

Revision 0: System Design

Group 8 - C.A.V.E: Cave Assessment and Visualization Equipment

Abdul Rahim Khan
Andrew Brink
Gokberk Yilmaz
Nicholas Trimble
Tabish Faisal

December 16, 2025

1 Introduction

The goal of this project is to create a portable device that can reliably and accurately reconstruct 3-D mappings of low light environments, such as crevice caves in the Niagra Escarpment, while avoiding the use of expensive technologies and solutions such as a 3-D LiDAR.

1.1 Purpose

This document will outline the specifications and design of the various modules within the device to achieve the requirements previously established. It will also partition these requirements into hardware and software sections and define the necessary interfaces and interactions between components.

1.2 Scope

The entirety of the device's design falls under the scope of this document, including the physical design, power delivery, data storage, and interactions between the portable recording device and the offline data processing algorithm. Hardware components will have their dimensions and electrical interfaces specified, and software components will be broken into modules to create well documented interfaces and hierarchical relationships.

1.3 Context Diagrams

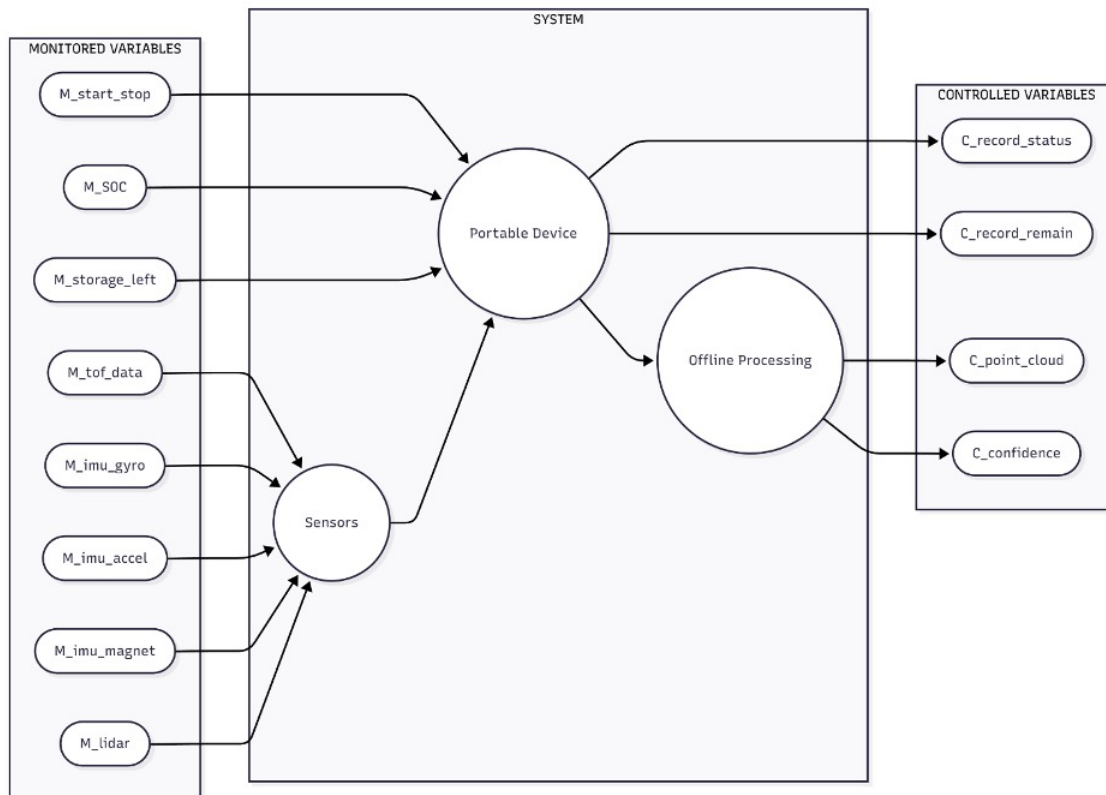


Figure 1: Representation of System Boundaries

1.4 Definitions

See System Requirements and Hazard Analysis section 2.1 for definitions relevant to this document.

2 System Hardware Components

2.1 Raspberry Pi 5

A Raspberry Pi 5 single board computer (SBC) will be used for collecting and temporarily storing the information captured by the sensors of the system.

2.1.1 Inputs and Outputs

Note that these are only within the context of the physical and electrical connections to the SBC.

Direction	Name	Format	Description
Input	Start button	GPIO	User control over recording (<i>M_start_stop</i>)
Input	ToF Data	USB 2.0	Series of point maps as captured by the time-of-flight sensor (<i>M_tof_data</i>)
Input	IMU	I2C	Various orientation data (<i>M_imu_gyro</i> , <i>M_imu_accel</i> , <i>M_imu_magnet</i>)
Input	2D LiDAR	TBD	Longer distance scans with a wider field-of-view (FOV) to aid in the point cloud merging (<i>M_lidar</i>)
Input	Power	USB-PD	5V, 5A via the RPi Head board - <i>Refer to Appendix A</i> . *Note that this is not USB-PD compliant
Output	Capture Data	TBD	Database of captured information from a recording session
Output	Recording status	LED or button toggle	Indicates whether the a device is recording (<i>C_record_status</i>)
Output	Remaining time	LEDs or display	Communicates recording time available to user (<i>C_record_remain</i>)

2.1.2 Behavior

Takes in power and the various sensor signals and routes them to the software running on the Pi's system-on-chip. The software will provide appropriate outputs to control the sensors and drive the various GPIO. This component acheives requirement RE1.

2.1.3 Dimensions

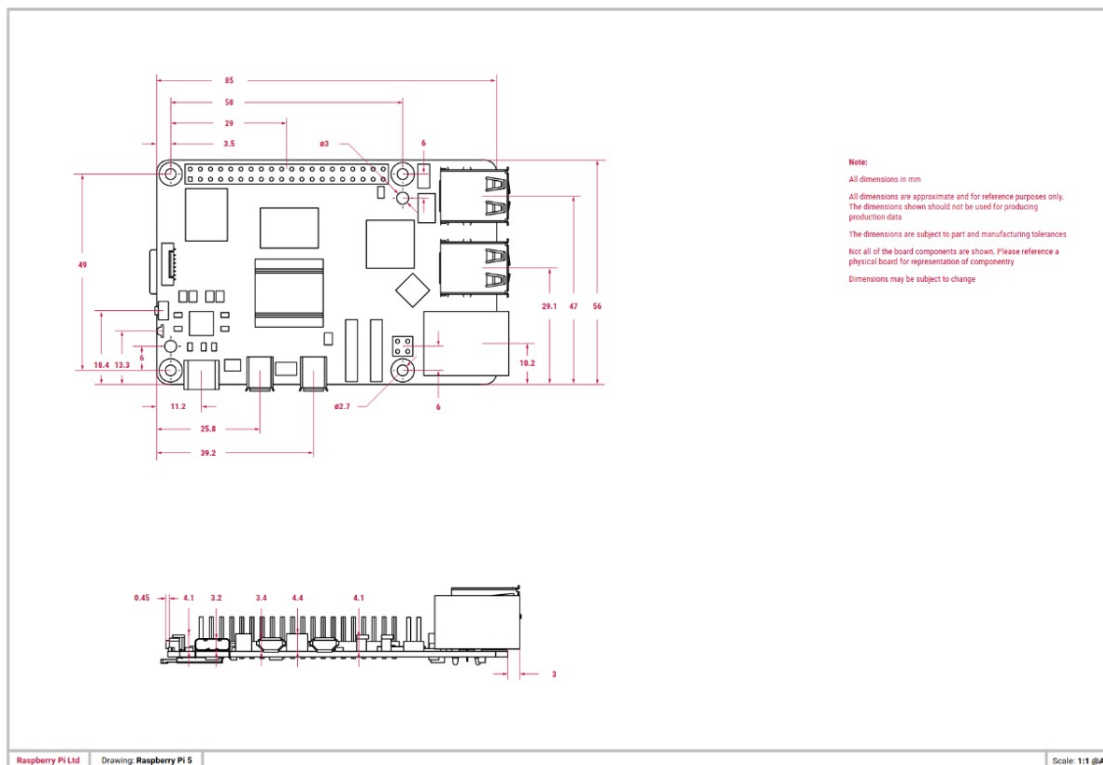


Figure 2: Dimensions of Raspberry Pi 5

2.1.4 Electrical Interfaces

Refer to Raspberry Pi RP1 Peripherals in section 8 (References).

2.2 ToF Sensor

The time-of-flight sensors used are SIPEED MaixSense A010 3D ToF sensors.

2.2.1 Inputs and Outputs

Direction	Name	Format	Description
Input	Power	USB-C	5V power from the Raspberry Pi
Input	ToF Camera Captures	940nm Laser	Depth information produced by the forward camera
Output	ToF Data	USB 2.0	3D point cloud, continuous scans (M_tof_data)

2.2.2 Behavior

This sensor measures the distance between itself and the environment around it and produces a series of point clouds representing a capture at a certain time instant. The interval between these

captures is to be controlled by the data collection software subsystem (S1-M1) and will follow requirements RT1 and RT2.

2.2.3 Dimensions

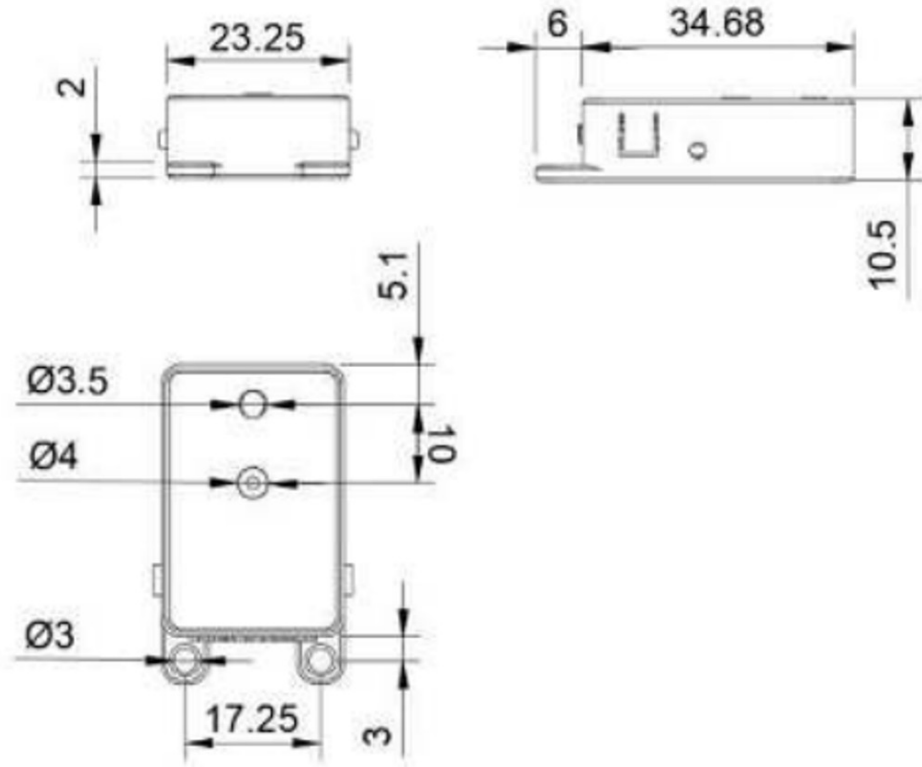


Figure 3: Dimensions of ToF sensor

2.2.4 Electrical Interfaces

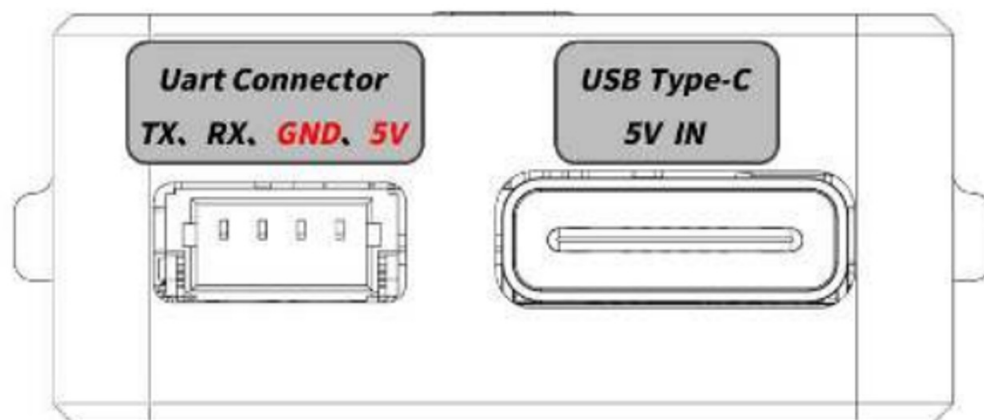


Figure 4: Electrical Interfaces of ToF sensor

2.3 IMU

This component is the Adafruit BNO055 Absolute Orientation Sensor.

2.3.1 Inputs and Outputs

Direction	Name	Format	Description
Input	User Motion	Movement in 3D space	Movement of the operator through the scanning area
Input	Power	5V	Input power from the IMU connector
Output	IMU data	I2C	Quaternion, linear acceleration, rotation, gravity, heading

2.3.2 Behavior

The IMU is a 9-axis sensor that will be used to determine displacement and rotation during scans. It produces fused data from a gyroscope, accelerometer, and geomagnetic sensor to produce matrices that represent displacements and rotations through the 3-D scanning area. Captures from this sensor will also be on an interval conforming to RT1 and RT2.

2.3.3 Dimensions

Measurements in this figure are in inches.

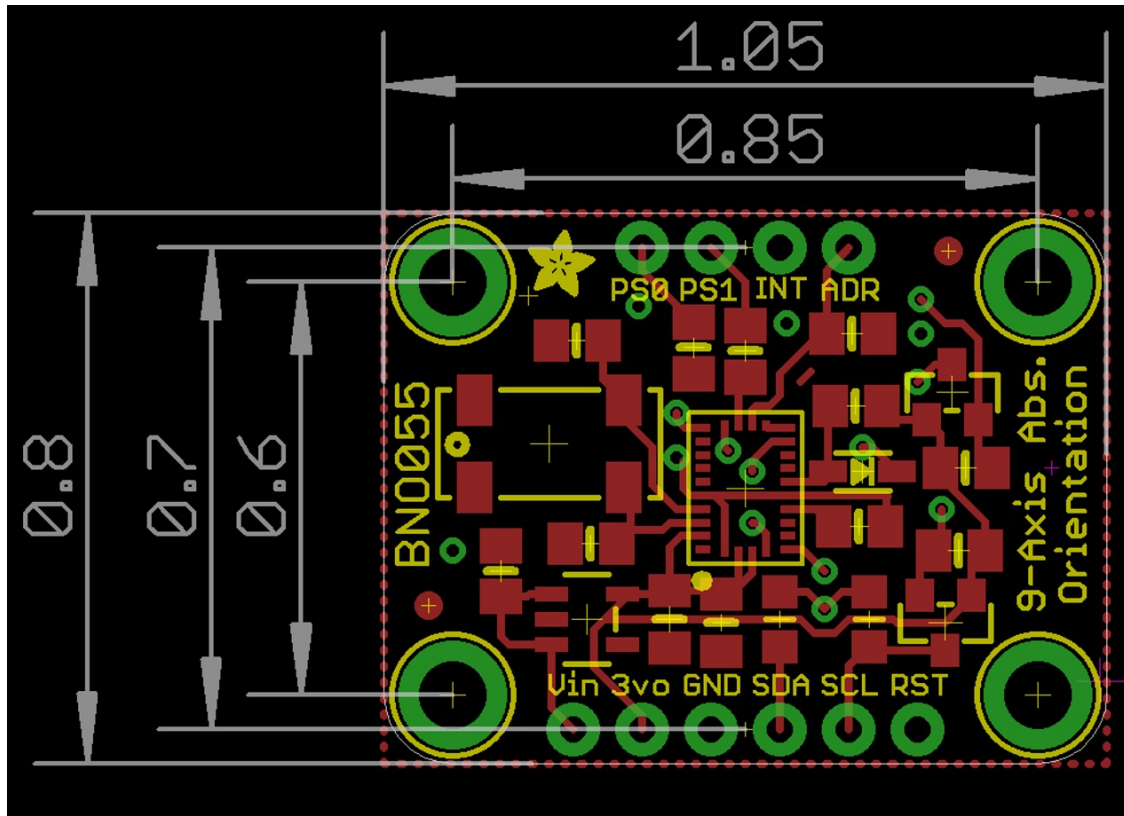


Figure 5: Dimensions of IMU

2.3.4 Electrical Interfaces

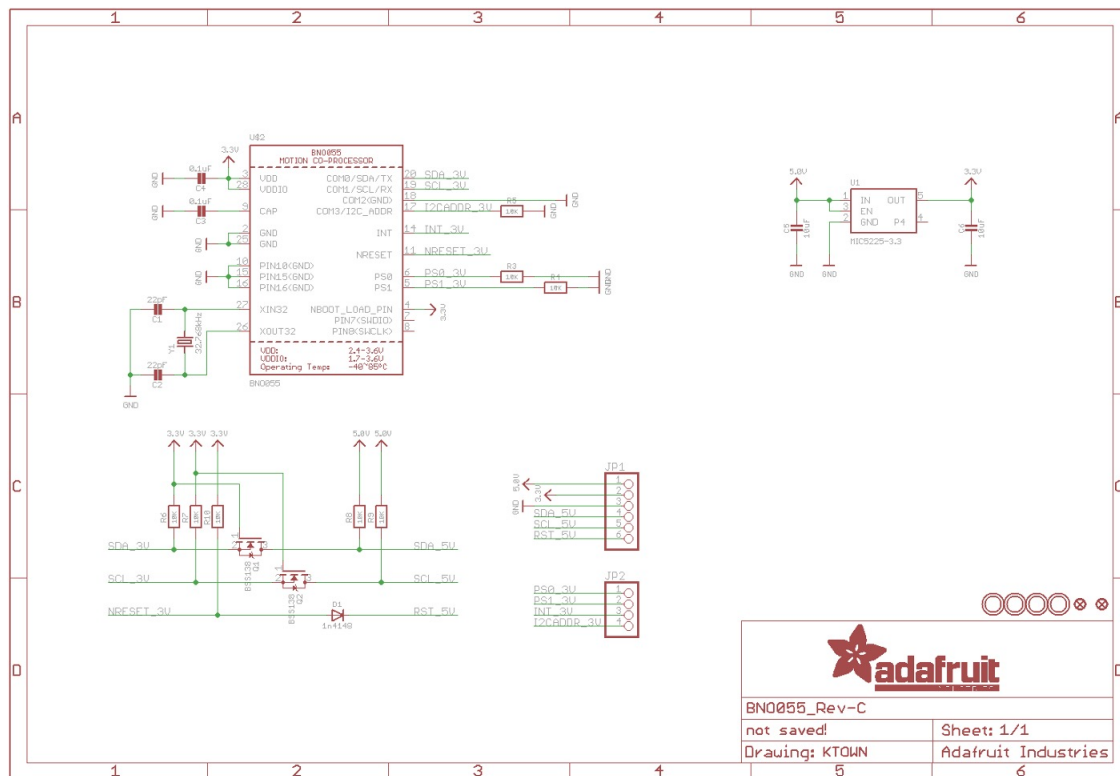


Figure 6: Electrical Interfaces of IMU

2.4 2D Li-DAR

This component is available from Studica as the T-Mini LiDAR Kit.

2.4.1 Inputs and Outputs

Direction	Name	Format	Description
Input	Laser Scan	Point Cloud	360 degree scan of a single 2-D plane at 6-12Hz, 0.05-12m range
Input	Power	5V	Input power from the Raspberry Pi 5
Output	LiDAR scan	UART/USB	Digital output of each scan over either USB via a converter or directly through UART

2.4.2 Behavior

Scans a 2-D slice by rotating 360 degrees with a range of 0.05-12m and 0.54deg resolution. Provides an additional point cloud that will be used to help merge the more detailed 3-D scans from the ToF sensors, improving c_confidence to conform to RB5.

2.4.3 Dimensions

Measures 38.6mm wide and long, and 33.9mm tall, plus cable.



Figure 7: Dimensions of 2D-LiDAR

2.4.4 Electrical Interfaces

Four pin JST-GH cable carrying 5V, UART Rx, UART Tx, and ground.

2.5 Battery Pack

The exact pack used may change depending on price, availability, and depending on whether tests with lower input voltages (15V) prove feasible. Current designs are based on the UGREEN Nexode 100W 3-Port 20,000mAh power bank as it provides full 20V PD at a reasonable price and size.

2.5.1 Inputs and Outputs

Direction	Name	Format	Description
Input	Charging	USB-PD	Recharging of the battery pack from the wall in between scanning sessions
Input	Power	USB-PD	Providing 20 or 15V power to the Rpi Head, then distributed to other devices

2.5.2 Behavior

This module is a standard USB-PD battery pack to provide safe, convenient, and portable power to the system. It will include a battery indicator to indicate to the user how much recording time is left and will be stored somewhere on the users' person rather than on the helmet to lower the risk of it being damaged by low ceilings. Relevant requirements are RB4 and RE2.

2.5.3 Dimensions

It will fit in a pocket or small backpack, with this pack measuring 140mm high by 80mm wide by 30mm wide.

2.5.4 Electrical Interfaces

Standard USB Type-C with at least USB 2.0 signalling to enable Power Delivery.

2.6 Helmet Mount

2.6.1 Inputs and Outputs

This module has no electrical signals or information to process and so has no inputs or outputs. From a mechanical standpoint its inputs are the masses of the TOF sensors, LiDAR, IMU, and Raspberry Pi, and its output is their combined masses loaded on the helmet.

2.6.2 Behavior

The purpose of this module is to provide a stable surface for the helmet mounted components to be secured to a caving helmet. Each module must be in a position where it can carry out uninterrupted scans (ie. TOF facing outward, LiDAR has full 360 line of sight). To enable the custom shapes needed to hold various sizes of sensors on a round platform the material will likely be 3-D printed PLA plastic. This material provides sufficient strength for the relatively small and light sensors and allows heat-set inserts to securely screw the each component to it. The primary requirement to conform to is RE2.

2.6.3 Dimensions

Final dimensions of the module are to be determined but they must keep the mounted modules as close to the helmet as reasonable, so they should not exceed the approximate 400x300x200mm cube around the user's head.

3 Software Subsystem S1: Data Collection

3.1 Module M1: Time-of-Flight (ToF) Sensor

3.1.1 Module Information

The ToF module is responsible for acquiring depth measurement data from the sensors and outputting time-stamped point cloud data. The module provides raw 3D point cloud frames in continuity that represents the surrounding environment. The hidden design decision in this module are the calibration parameters (i.e. serial access between sensor and the local system) and the calculation process from depth values to 3D point cloud maps.

3.1.2 Uses Hierarchy

Uses: ROS2 Library/Module

Used By: Module M5 (Data Aggregation)

3.1.3 MIS

The input and output of this module is given below:

- Input
 - Raw ToF sensor measurements from environment capture (depth values and intensity with RGB color mapping)
- Output
 - Time-stamped point cloud frames

3.1.4 MID

Methods:

- initialize_tof
 - Description: Establish serial connection from the ToF sensor to the connected system.
 - Parameters: None
 - Returns: Status flag whether the sensor is detected or not.
- publish_point_cloud
 - Description: Logs the calculated point cloud at a specified FPS.
 - Parameters: Depth-measurement data from ToF sensor
 - Returns: Continuous point cloud frames at 'x' FPS

3.1.5 Scheduling

The sensor data is read and publishes the point cloud frames in a set interval determined by the FPS value

3.2 Module M2: IMU

3.2.1 Module Information

The IMU provides inertial data to the Raspberry Pi, which will be used to improve the accuracy of our final model. This module reports the real-time rotation and acceleration data, which is logged for later use.

3.2.2 Uses Hierarchy

Uses: None

Used by: Module M3 (Data Aggregation)

3.2.3 MIS

- Input:
 - Analog inertial data from the environment
- Output:
 - Acceleration data, gyroscopic data, and linear acceleration data

3.2.4 MID

Methods:

- `sensor_acceleration`
 - Description: A getter function to read acceleration data from the BNO055 sensor over I2C
 - Parameters: None
 - Returns: Acceleration data
- `sensor_gyro`
 - Description: A getter function to read gyroscopic data from the BNO055 sensor over I2C
 - Parameters: None
 - Returns: Gyroscope data
- `sensor_linear_acceleration`
 - Description: A getter function to read linear acceleration data from the BNO055 sensor over I2C
 - Parameters: None
 - Returns: Linear acceleration data

3.2.5 Scheduling

The sensor data needs to be read every time step, but can be logged to non-volatile storage at any point during operation.

3.2.6 Initialization

The sensor is initialized over the I2C bus with the libraries provided by the manufacturer.

3.3 Module M3: Data Aggregation

3.3.1 Module Information

The data aggregation module is responsible for producing time-aligned sensor data and synchronize each ToF point cloud frame with the IMU data to the closest time ($\Delta_{t_IMU} - \Delta_{t_point_cloud}$) it was acquired.

3.3.2 Uses Hierarchy

Uses: Module M1 (ToF sensor) and Module M2 (IMU)

Used by: None

3.3.3 MIS

- Inputs:
 - Stream of ToF point cloud frames
 - Stream of IMU data
- Outputs:
 - Live data packet (ROS2 Node) containing time-matching point cloud and IMU data

3.3.4 MID

Methods:

- push_cloud_frame
 - Description: Insert new point cloud frame into a buffer along with it's timestamp
 - Parameters: Timestamped point cloud frame
 - Returns: Boolean (True for success)
- push_imu_data
 - Description: Insert new IMU data into an IMU buffer
 - Parameters: Timestamped IMU data
 - Returns: Boolean (true for success)
- sync_data
 - Description: Synchronizes the point cloud frame in buffer with the closest-time IMU data in the secondary
 - Parameters: None
 - Returns: Data packet containing matching point cloud frame and IMU data
- send_data_packet
 - Description: Outputs the recently synchronized data packet and clears out the buffers
 - Parameters: None
 - Returns: Synchronized data packet

3.3.5 Scheduling

This module is triggered everytime a point cloud frame or IMU data is received

3.4 Module M4: Rosbag Recorder

3.4.1 Module Information

This module records the synchronized sensor data packet and stores it as a rosbag file so that it can be used later in the point cloud processing.

3.4.2 Uses Hierarchy

Uses: Module M3 (Data Aggregation)

Used By: Module M5 (Data Ingest)

3.4.3 MIS

- Input:
 - Synchronized data packet
 - Storage path
- Output:
 - Rosbag file

3.4.4 MID

Methods:

- start_record
 - Description: Starts recording the synchronized data packet stream
 - Parameters: Data packet
 - Returns: Rosbag file containing continuous data packets
- check_rosbag_directory
 - Description: Checks whether directory/file exists for storing the rosbag file
 - Parameters: Path to directory/file
 - Returns: Boolean (0 if directory/file does not exist)

3.4.5 Scheduling

Starts upon user input

4 Software Subsystem S2: Point Cloud Processing

4.1 Module M5: Data Ingest

4.1.1 Module Information

The Data Ingest module is part of the Point Cloud Processing subsystem. The responsibility of this module is to take the data previously collected by the Data Collection subsystem, which is stored in the form of a rosbag, and prepare it for use by the other modules in this subsystem.

4.1.2 Uses Hierarchy

Uses: Module M4 (Rosbag Recorder)

Used by: Module M6 (Sensor Fusion)

4.1.3 MIS

- Input:
 - Rosbag file
- Outputs:
 - IMU messages
 - Data from ToF sensors
 - Data from 2D LiDAR sensor, if applicable

4.1.4 MID

Methods

- `set_rosbag_filename`
 - Description: Attempt to access file specified
 - Parameters: String representing filename
 - Returns: True if rosbag is valid
- `get_imu_message`
 - Parameters: none
 - Return: IMU data at a specific instant in time and timestamp, or indication of failure if no data is available.
- `get_tof_frame`
 - Parameters: index of tof camera to access
 - Return: Depth image (or point cloud) of data recorded by specified ToF camera with timestamp, or indication of failure if no data is available.
- `get_lidar_frame`
 - Parameters: none
 - Return: Point cloud of data captured by LiDAR with timestamp, or indication of failure if no data is available.

4.1.5 Scheduling

There are no scheduling requirements, as this subsystem does not operate in real time. The functions may be called whenever the other modules are ready for the data.

4.2 Module M6: Sensor Fusion

4.2.1 Module Information

This module uses the data from the IMU, ToF, and LiDAR sensors to build a 3D point cloud. The IMU data is integrated to provide an initial estimate of the transformation between two ToF captures. This estimate is used to perform point cloud alignment for each new point cloud produced by the ToF cameras. Finally, the LiDAR data is added to the point cloud, to increase the range.

4.2.2 Uses Hierarchy

Uses: Module M5 (Data Ingest)

Used by: None

4.2.3 MIS

- Inputs

- IMU data
- ToF data
- LiDAR data

- Outputs

- 3D point cloud

Methods

- update_imu

- Description: Integrates new IMU data to estimate transformation
- Input: imu data
- Output: transformation from previous update_tof time till now

- update_tof

- Description: Using transformation estimated by IMU as initial guess, align point clouds
- Input: new point cloud that needs to be registered
- Output: transformation from previous point cloud to new point cloud

- combine_tof

- Description: Combines point clouds from the two tof sensor, so that they can be treated as one for update_tof step
- Input: two different point clouds with same timestamp from separate cameras
- Output: one point cloud

- update_lidar

- Description: Add points measured by LiDAR sensor to full point cloud data member
- Input: point cloud from LiDAR (2D)

- `get_point_cloud`
 - Description: Get the full point cloud with all data that has been processed so far
 - Output: full point cloud.

4.2.4 MID

Private data members

- `point_cloud` → Collection of points in 3D space which represent all the data that has been processed so far
- `imu_state` → Pose (position and orientation) of IMU, with 1st and 2nd derivatives (velocity and acceleration), to be updated by Kalman filter
- `imu_state_covariance` → Covariance matrix corresponding to `imu_state`, used by Kalman filter.
- `tof_transforms` → List or key-value structure containing timestamps and transformations as calculated by `update_tof`

Design of methods

- `update_imu` → A Kalman Filter (or a variation such as the Error State Kalman filter) is used to predict the movement of the helmet, using acceleration, gyroscope, and magnetometer data.
- `update_tof` → Use Iterative Closest Point (ICP) algorithm, using an implementation such as that in Point Cloud Library (PCL), to find the transformation between the previous ToF sensor point cloud and the new one. This transformation, together with the timestamp, is stored in `tof_transforms`. The estimate of the transformation which has been obtained using `update_imu` is used as the initial guess for the ICP algorithm.
- `combine_tof` → If the two point clouds have the same timestamp, append the points from one point cloud to the other point cloud.
- `update_lidar` → Add the LiDAR points to `point_cloud`, using the transformation matrix that has been calculated for the current timestamp from the `update_tof` method. The use of ICP is not required or helpful here, because the points are from a 2D slice which is moving through space in 3D.
- `get_point_cloud` → Returns copy of `point_cloud` private data member

4.2.5 Notes

The paper “Perception in the Dark” has been used as reference for how the Error State Kalman filter and ToF ICP can be implemented, and their open-source code will be used as a reference as well. This work is limited to one ToF sensor, however, and does not use 2D lidar. (not sure how to cite this properly but here is a link: <https://doi.org/10.3390/s20051263>)

5 Normal Operation

The normal operation would include the following steps:

- Power the system on
- Allow ample time for required start-up sequences
- There will be an indication as to when scanning begins
- Scanning will continue until you manually stop it
- Once scanning ends, it will process and produce the 3-D image of the scan
- After we are done using the system, we may choose to either do another scan, or to turn off the system.

6 Undesired Event Handling

Undesired events can occur at each major step and in between them,

- During turning the system on:
 - The system may not turn on
 - * In this case, we'd check if we properly turned it on,
 - * If that does not work, we'd check if the battery is charged
 - * If it still does not work, then we'd check any wiring
 - The system may turn on but then crash/turn off
 - * In this case, this may be a software or sensor issue, in which case we'd have to determine the root cause.
- While we wait for the start-up sequences
 - Previous iterations may try to start up again
 - * Make sure all instances are closed during shut-down so that it wouldn't restart
 - Specific parts may fail or have an error
 - * Using well-defined error/loading messages to determine which part of the system particularly is giving issues, and work on countermeasures accordingly
 - The battery may suddenly run out
 - * If this happens, ensure proper shutdown protocol so even if it suddenly loses power, it wouldn't have devastating consequences
- When the scanning begins,
 - The battery may run out during scanning
 - * Since the scan is stored in RAM, the ongoing scan will be lost, however
 - The indicator doesn't indicate correctly
 - * Have a second indicator just in case → if this fails, then have to check the entire system

- * Ensure it isn't a human error, if not then have to check the system for the fault
- Sudden/erratic movement for scanning
 - * Correct training on handling.
 - * Try and handle it by having “blank” space instead of warped space.
 - * Well timed snapshots to ensure minimal loss/warping.
 - * In extreme cases, erasure of the entire scan.
- When scanning ends,
 - Manual stopping may not work
 - * Have a secondary method to manually stop the scan.
 - * Allow scans to save dynamically if there is a sudden shut-down, allowing us to in extreme cases disconnect whilst retaining information .
 - Automatic stopping may not work
 - * Have a visual indicator for time left in the scanner, to ensure it was supposed to stop.
 - * Override automatic stopping with manual stopping.
 - Scan may be lost
 - * Store the frames dynamically in such a way where when a sudden issue occurs, it will keep it safe.
 - * Have backup/concurrent instances saved in case one fails.
 - Battery runs out
 - * Make sure most progress won't be lost. It is okay if the algorithm breaks suddenly, but it shouldn't affect the frames
- Producing the 3-D image of the scan,
 - Battery runs out
 - * Have a buffer of raw processed files so that it can reproduce the image if needed.
- When it comes to repeating the scan or turning off,
 - Doesn't turn off
 - * Allow a secondary override to forcefully shut down the system without catastrophic failure.
 - The repeat uses the same data as before
 - * Make sure shared variables are proper initialized and set to the default value before starting up.
 - * This can be done through double-checking via validation.

7 Appendix A: Sandmaze PCBs

Last years' Sandmaze group created several PCBs which this group will build off to better integrate the data collection subsystem.

7.1 Rpi Head

7.1.1 Inputs and Outputs

Direction	Name	Format	Description
Input	Start button	USB Type-C	Power and data connection to the IMU
Input	ToF Data	USB 2.0	Series of point maps as captured by the time-of-flight sensor (M_tof_data)
Input	PD Input	USB-PD	Standard 20V USB PD sink
Input	User Buttons	SMT side mounted buttons (U6/8)	Allows user actuation of GPIO pins- pull down connection
Output	RPi GPIO	2x20 pin 2.54mm pitch header	IMU and button connections to Raspberry Pi SBC
Output	Pi Power	USB Type-C	5V5A power for RPi5, connecting to 'Fake Wire'
Output	User LEDs	SMT LEDs	Display status of power conversion, some connected to GPIO (LED1/2, U10)

7.1.2 Behaviour

This board connects on top of the Raspberry Pi 5 as a 'hat' device to break out some of the GPIO pins into more useful forms and provide power from the battery. The most important function it performs is to buck-convert the standard USB-PD power from the battery pack into the non-standard 5V5A supply required by the Pi 5 to avoid voltage drops in high load scenarios. The USB-PD spec only allows up to 3A to be supplied at 5V profiles, necessitating this conversion. It also provides a connection for the IMU Connector's USB Type-C cable which is then attached to the I2C GPIO pins. There are also buttons and LEDs attached to GPIO pins.

7.1.3 Dimensions

The board is the same dimensions as a Raspberry Pi (see section 2.1), with mounting holes in the same positions to allow it to be securely stacked on top of the Pi.

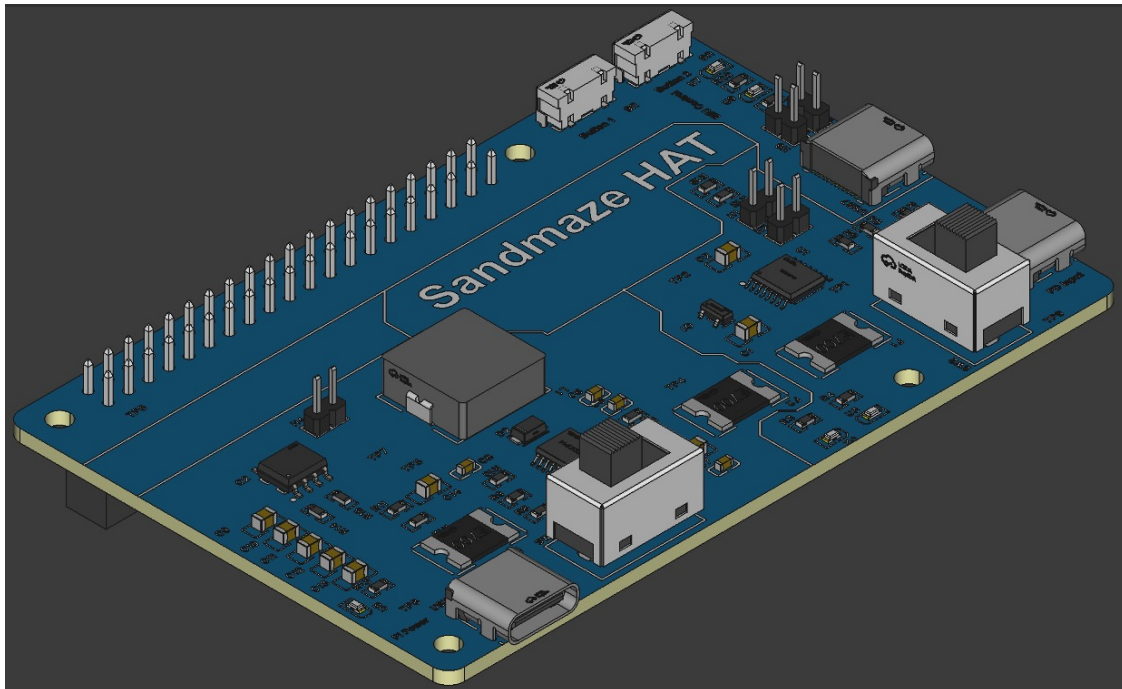


Figure 8: Dimensions of Sandmaze HAT

7.1.4 Electrical Interfaces

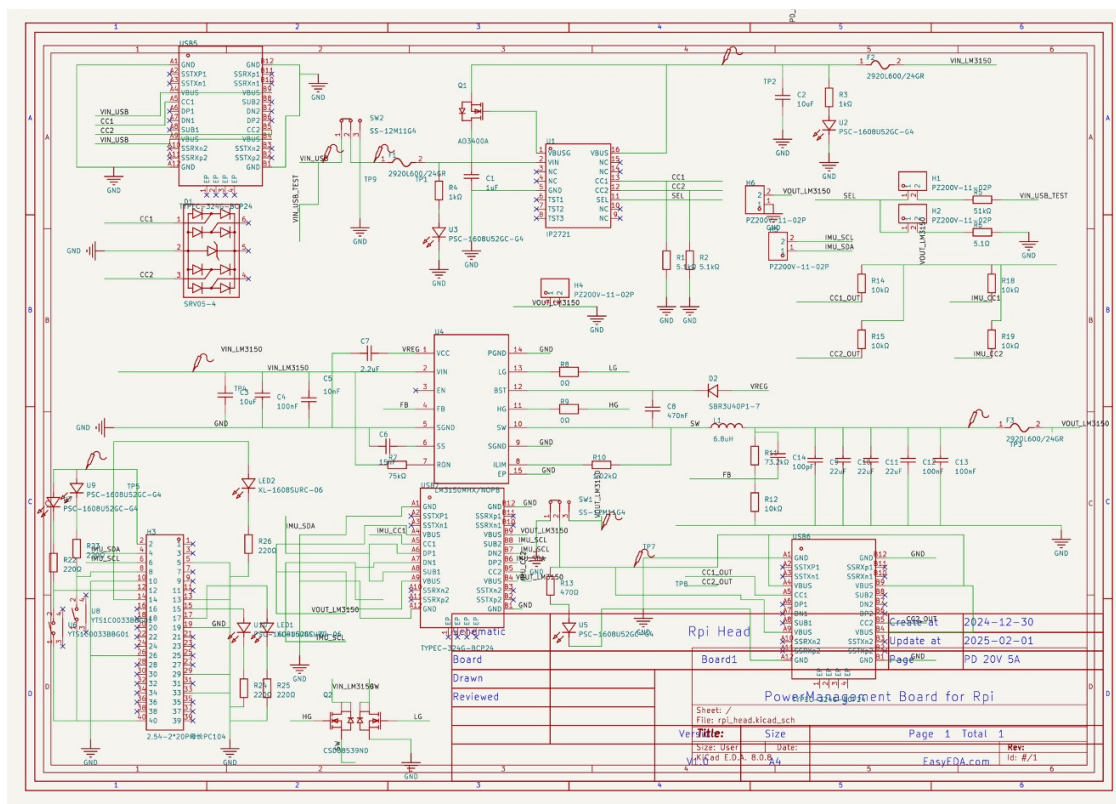


Figure 9: Electrical Interfaces of Sandmaze HAT

7.2 IMU Connector

7.2.1 Inputs and Outputs

Direction	Name	Format	Description
Input	IMU header	2.54mm header pins	Power and data connection to the IMU
Output	IMU USB	USB Type-C	IMU wires re-pinned to use Type-C

7.2.2 Behaviour

This board passes through the connections from the IMU board, converting the connector from a 1x6 pin 2.54mm pitch pin header to a standard USB Type-C connector.

7.2.3 Dimensions

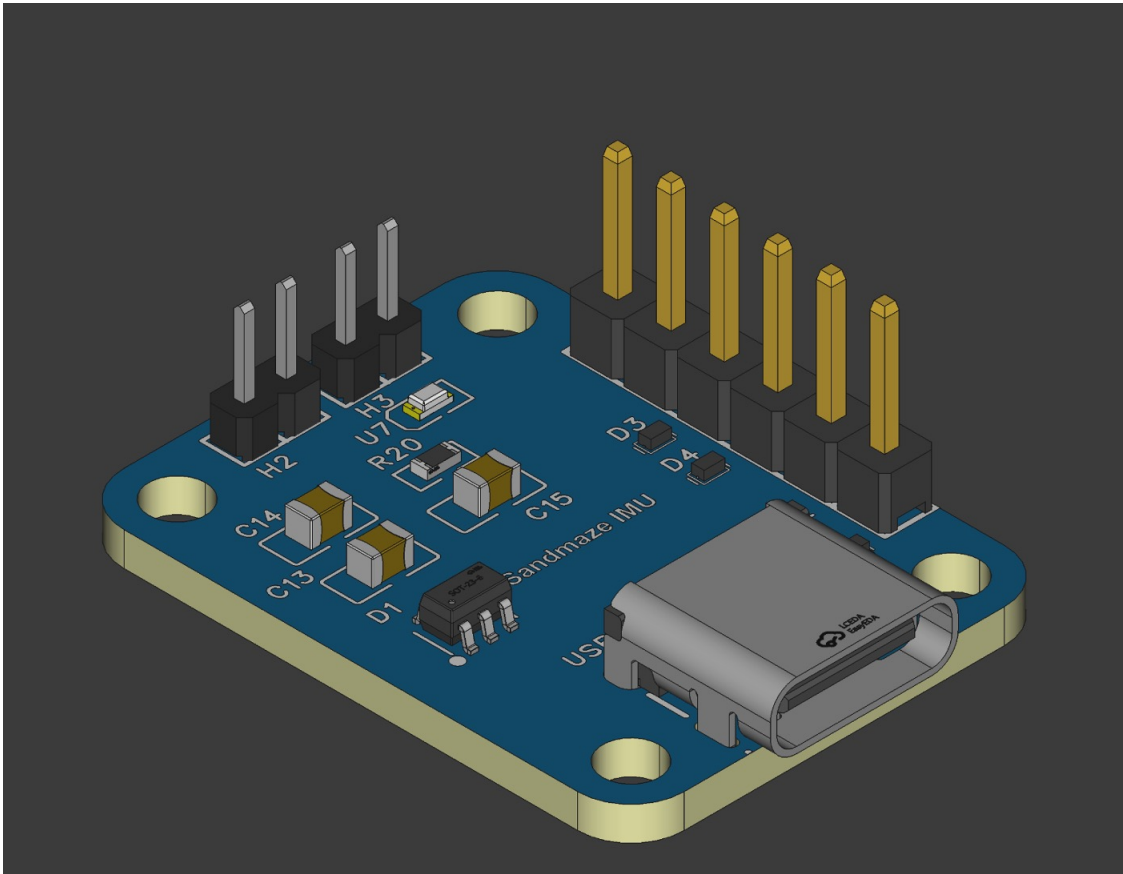


Figure 10: Dimensions of IMU connector

7.2.4 Electrical Interfaces

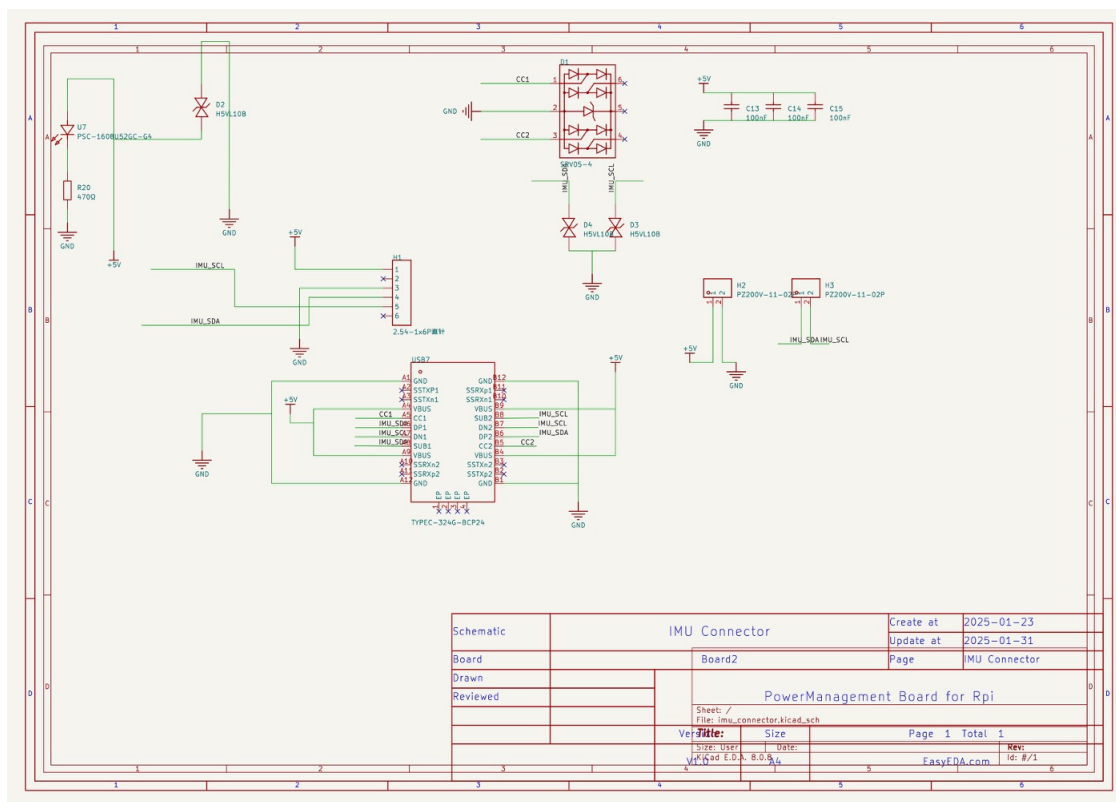


Figure 11: Electrical Interfaces of IMU connector

Pin	Purpose	Pin	Purpose
H1-1	5V to IMU	USB7-VBUS	5V from Pi
H1-3	GND for IMU	USB7-GND	GND to Pi
H1-4	IMU SDA	USB7-DN*	IMU SCL
H1-5	IMU SDL	USB7-DP*	IMU SDA

Direction	Name	Format	Description
Input	IMU header	2.54mm header pins	Power and data connection to the IMU
Output	IMU USB	USB Type-C	IMU wires re-pinned to use Type-C

7.3 ‘Fake Wire’ USB Type-C extension

7.3.1 Inputs and Outputs

Direction	Name	Format	Description
Input	RPi Head power	USB Type-C	5V5A power from the RPi Head

7.3.2 Behaviour

This is a pure passthrough acting as a short cable between the RPi Head's power output port and the power input on the Raspberry Pi 5.

7.3.3 Dimensions

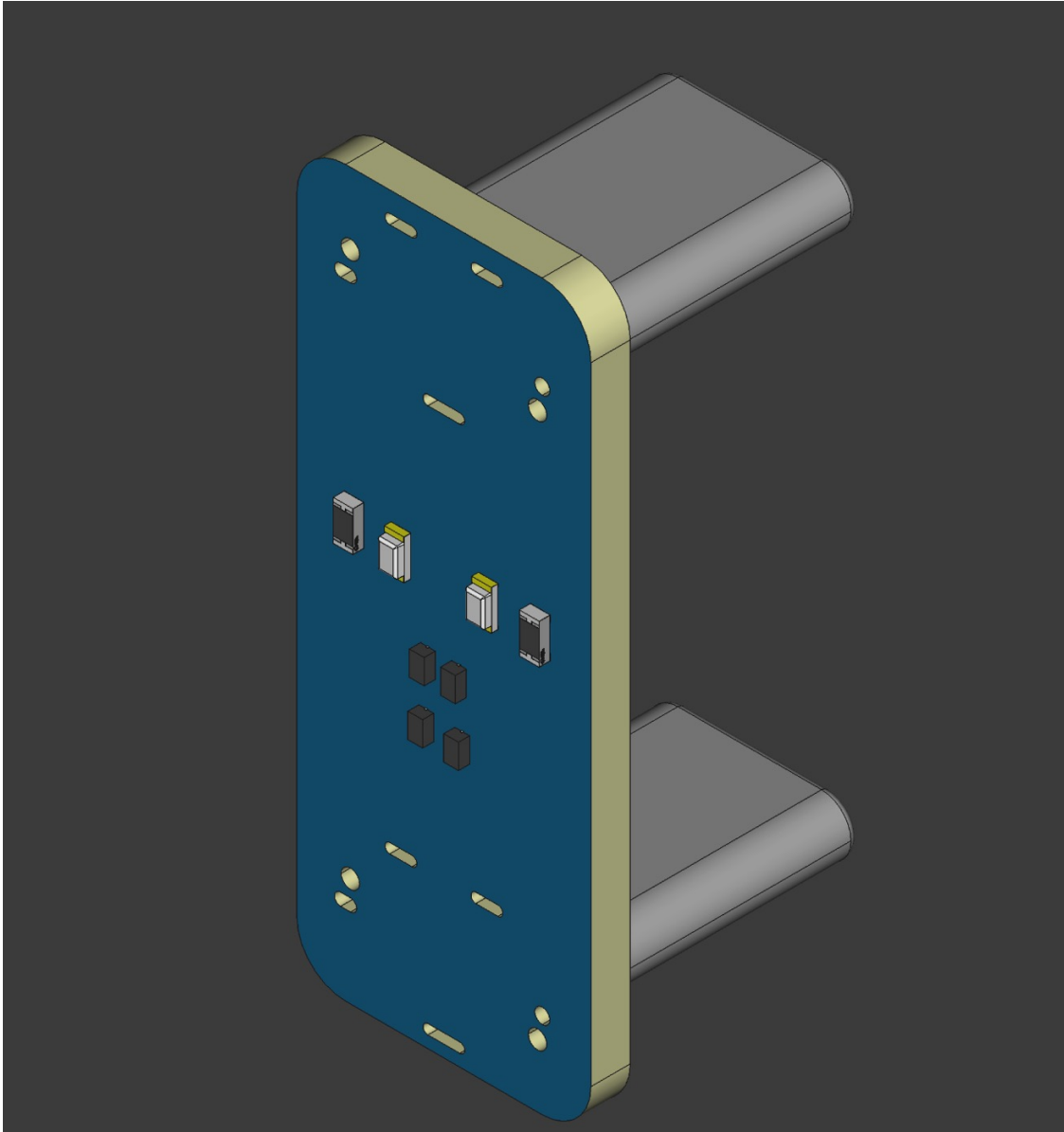


Figure 12: Dimensions of Fake Usb C

7.3.4 Electrical Interfaces

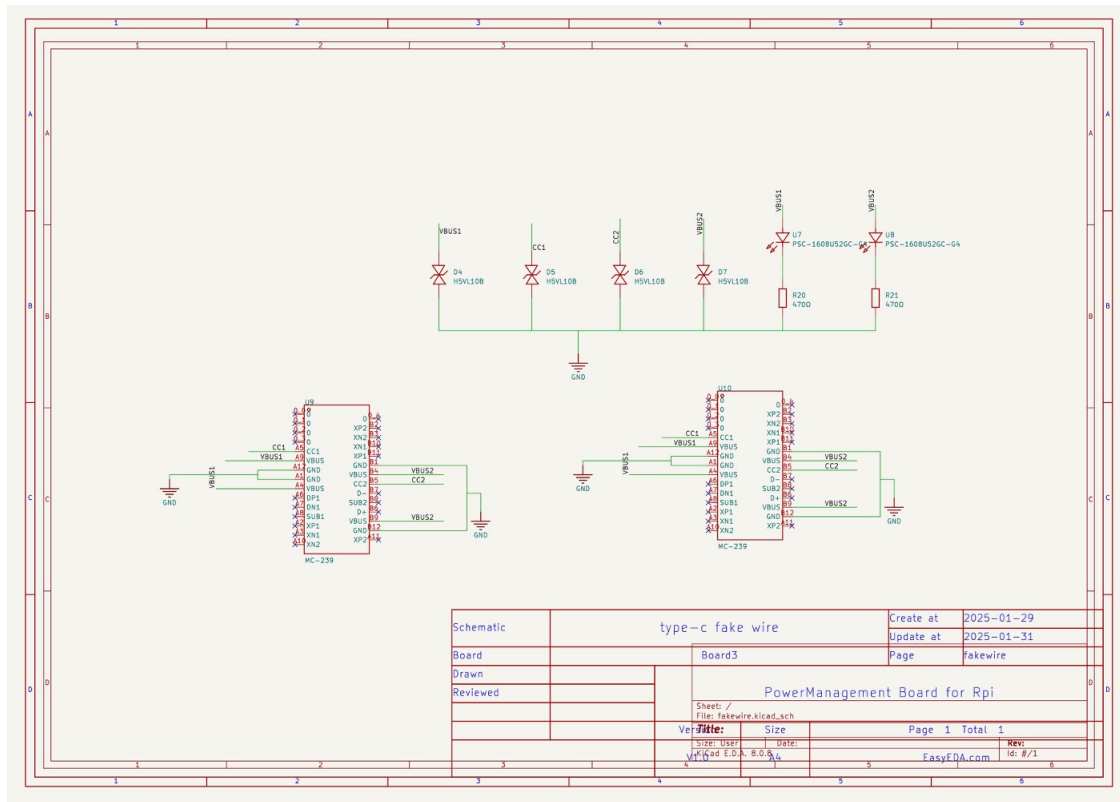


Figure 13: Electrical Interfaces of Fake Usb C

Pin	Purpose
U*-VBUS	5V, 5A power rail
U*-GND	Electrical ground
CC1	Configuration Channel for PD negotiation
CC2	See above

8 References

Raspberry Pi mechanical drawing:

<https://pip-assets.raspberrypi.com/categories/892-raspberry-pi-5/documents/RP-008347-DS-1-raspberry-pi-5-mechanical-drawing.pdf?disposition=inline>

Raspberry Pi RP1 Peripherals guide:

<https://pip-assets.raspberrypi.com/categories/892-raspberry-pi-5/documents/RP-008370-DS-1-rp1-peripherals.pdf?disposition=inline>

ToF sensors datasheet:

https://dl.sipeed.com/shareURL/MaixSense/MaixSense_A010

IMU datasheet:

<https://learn.adafruit.com/adafruit-bno055-absolute-orientation-sensor/downloads>

LiDAR product page:

<https://www.ydlidar.com/product/ydlidar-t-mini-plus>

USB battery bank:

<https://www.staples.ca/products/3119320-en-ugreen-nexode-100w-3-port-20000-mah-power-bank-portable-charger-silver>